Day 12: React Routing

Multi-page Student Portal with React Router

1. Topics Covered

- Introduction to React Router
- BrowserRouter
- Routes and Route
- Link and NavLink
- Navigation between pages
- Dynamic route parameters
- Basic 404 page

2. Task

- Install and configure React Router.
- Create Home, About, Students, and Contact pages.
- Navigate between pages without a full page reload.
- Create a basic not-found page.

Expected output: a multi-page React application using client-side routing.

3. Concepts Explained

Client-side routing. A normal website asks the server for a new HTML page on every link click. In a single-page app the browser loads one page once, and React Router changes what is shown based on the URL, so moving between pages is instant and the browser never reloads.

BrowserRouter. Wraps the whole app (in `main.jsx`) and lets React Router read and update the browser address bar.

Routes and Route. `Routes` holds the list of pages. Each `Route` maps a URL `path` to a component given in `element`. A route with `index` is the default child page, and `path="*"` matches any URL no other route matched, which makes it the 404 page.

Link and NavLink. `Link` replaces the HTML `<a>` tag and navigates without reloading. `NavLink` is a Link that knows whether it matches the current URL, so it can style the active page. The `end` prop stops the Home link (`/`) from staying active on every page.

Nested routes and Outlet. The `Layout` component holds the navbar and an `<Outlet />`. The navbar stays on screen while the Outlet is replaced by the page that matches the URL.

Dynamic route parameters. A path such as `students/:id` matches `/students/1`, `/students/2` and so on. The `useParams()` hook returns the value (`id`), which the component uses to find the right student.

useNavigate. Navigates from code instead of a link, for example after a button click: `navigate("/students")`.

4. Routes in This Project

URL	Component	Description
`/`	Home	Welcome page with a button to the students list.
`/about`	About	Short description of the project.
`/students`	Students	List of students; each has a View details link.
`/students/:id`	StudentDetail	Details of one student, found from the id in the URL. Shows a message if the id does not exist.
`/contact`	Contact	Controlled contact form with a success message.
`*`	NotFound	404 page for any unmatched URL.

5. Project Structure

```
router-app/
  index.html
  package.json
  vite.config.js
  src/
    main.jsx     (BrowserRouter wraps the app)
    App.jsx      (Layout, pages and all Routes)
```

6. How to Run

Requires Node.js. Check with `node -v`. Create the project and install React Router:

```
cd C:\dev
mkdir router-app
cd router-app
mkdir src
npm init -y
npm pkg set type=module
npm pkg set scripts.dev=vite
npm install react react-dom react-router-dom
npm install -D vite @vitejs/plugin-react
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Portal</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({ plugins: [react()] })
```

src/main.jsx

```
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Save the code in section 7 as `src/App.jsx`, then run `npm run dev` and open `http://localhost:5173`.

7. Source Code: src/App.jsx

```
import { useState } from "react";
import { Routes, Route, Link, NavLink, Outlet, useParams, useNavigate } from "react-router-dom";

// Sample data used by the Students pages
const STUDENTS = [
  { id: 1, name: "Aarav Sharma", email: "aarav@example.com", course: "Web Development" },
  { id: 2, name: "Diya Patil", email: "diya@example.com", course: "Data Science" },
  { id: 3, name: "Rohan Deshmukh", email: "rohan@example.com", course: "UI/UX Design" },
  { id: 4, name: "Sneha Kulkarni", email: "sneha@example.com", course: "Cyber Security" },
];

// Shared layout: navbar stays on screen, <Outlet /> swaps the page below it
function Layout() {
  const linkClass = ({ isActive }) => (isActive ? "nav-link active" : "nav-link");
  return (
    <>
      <style>{css}</style>
      <header className="navbar">
        <span className="brand">Student Portal</span>
        <nav>
          {/* NavLink adds the "active" state automatically; end makes "/" match only the home
              page */}
          <NavLink to="/" end className={linkClass}>Home</NavLink>
          <NavLink to="/about" className={linkClass}>About</NavLink>
          <NavLink to="/students" className={linkClass}>Students</NavLink>
          <NavLink to="/contact" className={linkClass}>Contact</NavLink>
        </nav>
      </header>
      <main className="content">
        <Outlet />
      </main>
    </>
  );
}

function Home() {
  return (
    <section>
      <h1>Welcome to the Student Portal</h1>
      <p>This is a multi-page React app built with React Router. Moving between pages never
          reloads the browser.</p>
      <Link className="button" to="/students">View students</Link>
    </section>
  );
}

function About() {
  return (
    <section>
      <h1>About</h1>
      <p>This project was built during the Technosignia internship to practise client-side
          routing: BrowserRouter, Routes, Route, Link, NavLink, route parameters and a 404
          page.</p>
    </section>
  );
}

function Students() {
  return (
    <section>
      <h1>Students</h1>
      <p className="muted">Select a student to open their details page.</p>
      <ul className="list">
        {STUDENTS.map((s) => (
          <li key={s.id} className="card">
            <div>
              <strong>{s.name}</strong>
              <div className="muted">{s.course}</div>
            </div>
            {/* Dynamic URL: /students/1, /students/2 ... */}
            <Link to={`/students/${s.id}`}>View details</Link>
          </li>
        ))}
      </ul>
    </section>
  );
```

```
}

function StudentDetail() {
  const { id } = useParams();          // reads :id from the URL
  const navigate = useNavigate();      // navigate from code
  const student = STUDENTS.find((s) => s.id === Number(id));

  if (!student) {
    return (
      <section>
        <h1>Student not found</h1>
        <p>There is no student with ID {id}.</p>
        <Link to="/students">Back to students</Link>
      </section>
    );
  }

  return (
    <section>
      <h1>{student.name}</h1>
      <dl className="details">
        <dt>Student ID</dt><dd>{student.id}</dd>
        <dt>Email</dt><dd>{student.email}</dd>
        <dt>Course</dt><dd>{student.course}</dd>
      </dl>
      <button className="button" onClick={() => navigate("/students")}>Back to students</button>
    </section>
  );
}

function Contact() {
  const [form, setForm] = useState({ name: "", message: "" });
  const [sent, setSent] = useState(false);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    setSent(false);
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    setSent(true);
    setForm({ name: "", message: "" });
  };

  return (
    <section>
      <h1>Contact</h1>
      {sent && <div className="success" role="status">Thanks, your message was sent.</div>}
      <form onSubmit={handleSubmit} className="form">
        <label htmlFor="name">Name</label>
        <input id="name" name="name" value={form.name} onChange={handleChange} required />
        <label htmlFor="message">Message</label>
        <textarea id="message" name="message" rows="4" value={form.message}
            onChange={handleChange} required />
        <button className="button" type="submit">Send message</button>
      </form>
    </section>
  );
}

function NotFound() {
  return (
    <section className="center">
      <h1>404</h1>
      <p>This page does not exist.</p>
      <Link className="button" to="/">Go to home</Link>
    </section>
  );
}

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="about" element={<About />} />
        <Route path="students" element={<Students />} />
        <Route path="students/:id" element={<StudentDetail />} />
        <Route path="contact" element={<Contact />} />
```

```
        {/* "*" matches any URL that no other route matched */}
        <Route path="*" element={<NotFound />} />
      </Route>
    </Routes>
  );
}

const css = `
  body { margin: 0; background: #eef2f6; font-family: system-ui, sans-serif; color: #1c2733; }
  .navbar { display: flex; flex-wrap: wrap; align-items: center; justify-content: space-
      between; gap: 12px; background: #1f5fa8; padding: 12px 24px; }
  .brand { color: #fff; font-weight: 700; font-size: 18px; }
  nav { display: flex; gap: 6px; flex-wrap: wrap; }
  .nav-link { color: #dbe8f8; text-decoration: none; padding: 8px 14px; border-radius: 8px;
      font-size: 15px; }
  .nav-link:hover { background: rgba(255,255,255,.15); }
  .nav-link.active { background: #fff; color: #1f5fa8; font-weight: 600; }
  .content { max-width: 760px; margin: 24px auto; background: #fff; border: 1px solid #d7dee6;
      border-radius: 12px; padding: 28px; }
  h1 { margin: 0 0 12px; font-size: 26px; }
  p { line-height: 1.6; }
  .muted { color: #5b6b7b; font-size: 14px; }
  a { color: #1f5fa8; }
  .button { display: inline-block; margin-top: 8px; padding: 10px 18px; background: #1f5fa8;
      color: #fff; border: 0; border-radius: 8px; font-size: 15px; font-weight: 600; text-
      decoration: none; cursor: pointer; }
  .button:hover { background: #174b87; }
  .list { list-style: none; padding: 0; margin: 16px 0 0; display: grid; gap: 10px; }
  .card { display: flex; justify-content: space-between; align-items: center; gap: 12px;
      padding: 14px; border: 1px solid #d7dee6; border-radius: 8px; }
  .details { display: grid; grid-template-columns: 120px 1fr; gap: 8px 12px; margin: 16px 0; }
  .details dt { font-weight: 600; } .details dd { margin: 0; }
  .form { display: grid; gap: 8px; }
  label { font-weight: 600; font-size: 14px; }
  input, textarea { padding: 10px 12px; font-size: 15px; border: 1px solid #b8c4d0; border-
      radius: 8px; font-family: inherit; }
  .success { background: #e3f4e8; color: #1e6b3a; padding: 10px 12px; border-radius: 8px;
      margin-bottom: 12px; }
  .center { text-align: center; }
  .center h1 { font-size: 64px; }
`;
```

8. Testing Checklist

- Click Home, About, Students and Contact: the page changes instantly with no browser refresh, and the current tab is highlighted.
- On Students, click View details: the address becomes `/students/1` and that student's details appear.
- Click Back to students: you return to the list.
- Open `/students/99`: a Student not found message appears.
- Open `/anything`: the 404 page appears, with a link back to Home.
- Submit the Contact form: a success message appears and the fields clear.

9. Result

The application is a multi-page React app using client-side routing. Pages switch without reloading the browser, student details load through a dynamic route parameter, and unknown URLs show a 404 page.